Exercise 1 – Column Pruning, Predicate Pushdown, and Compound Speed-up

A data engineering team stores a 1 TB clickstream dataset in two formats:

- **CSV** (row-oriented, gzip): compression ratio $r_{\text{row}} = 3$.
- **Parquet+Snappy** (columnar): compression ratio $r_{\text{col}} = 6$.

The schema has $n = 25$ columns of average width $w = 16$ bytes per row, so the row width is $W = 400$ bytes. The dataset contains $N = 2.5 \times 10^9$ rows. The Parquet file uses row groups of $G = 100{,}000$ rows.

Part A. Column pruning. A typical analytics query reads only $|S| = 5$ columns out of 25 to compute a session-level aggregate.

(a) Compute the bytes read by a CSV scan (compressed).

(b) Compute the bytes read by a Parquet scan with column pruning but *no* predicate pushdown.

(c) Compute the column-pruning ratio ρ_{prune}.

(d) Compute the speed-up of Parquet over CSV from column pruning and compression alone.

(a) The compressed bytes read is

$$B_{\text{row}}^{\text{comp}} = \frac{N \cdot W}{r_{\text{row}}} = \frac{1\,TB}{3} \simeq 333.33\,GB$$

(b) Reading 5 columns of 16 bytes each means reading $5 \times 16 = 80$ bytes per row

$$B_{\text{col}}^{\text{comp}}(S) = \frac{N \cdot \sum w_j}{r_{\text{col}}} = \frac{2.5 \times 10^9 \times 80}{6} = \frac{200\,GB}{6} \simeq 33.33\,GB$$

(c) $\rho_{\text{prune}} = 1 - \frac{\sum_{j \in S} w_j}{W} = 1 - \frac{80}{400} = 1 - 0.2 = 0.8$

(d) $\text{Speed-up} = \frac{B_{\text{row}}^{\text{comp}}}{B_{\text{col}}^{\text{comp}}} = \frac{333.33}{33.33} = 10\times$

Part B. Predicate pushdown. The same query also has a filter `WHERE country_code = 'FR'`. Out of all rows, $\sigma = 1/50$ have `country_code = 'FR'`, and country codes are randomly distributed across row groups (no sorting).

(e) Compute the probability that a single row group is *skipped* thanks to its min/max statistics. Comment.

(f) How many row groups are there in total?

(g) Now suppose the data is sorted by `country_code` before writing. What fraction of row groups must be read in this case? (Hint: contiguous matching rows.)

(h) Compute the bytes read by Parquet with column pruning + sorted pushdown. Compare to part (b).

(e) $P(\text{skip}) = (1-\sigma)^G = (1-0.2)^{100000} = 0.98^{100000} \approx 0$

Because the data is randomly distributed. Every row group of 100000 rows is practically guaranteed to contain at least one row with 'FR'

Therefore, predicate pushdown provides zero benefit here.

(f) $R = \frac{N}{G} = \frac{2.5 \times 10^9}{100000} = 25000$ row groups

(g) Fraction read $\approx \alpha = 1/50 = 2\%$

(h) Bytes read $\approx 33.33\,\text{GB} \times 0.02 = 0.667\,\text{GB}$

Comparison : it reads 50× less data than in part (b) because 98% of the row groups are entirely skipped.

Part C. Total speed-up.

(i) Compute the total speed-up of (sorted) Parquet over CSV for this query. Express as a multiplicative factor.

$$\text{Total speed-up} = \frac{\text{CSV bytes read}}{\text{Sorted Parquet bytes read}} = \frac{333.33\,\text{GB}}{0.667\,\text{GB}} = 500\times$$

Exercise 2 – Partition Design and the Small-File Problem

A retail company stores transaction logs in S3 as a Parquet data lake. The data is partitioned by

`country / year / month / day`

with the following cardinalities:

$$c_1 = 12 \text{ countries}, \quad c_2 = 5 \text{ years}, \quad c_3 = 12 \text{ months}, \quad c_4 = 31 \text{ days/month}.$$

Each day-leaf directory holds an average of 2 GB of compressed data, currently split into 200 files of average size 10 MB each.

Part A. Partition design.

(a) Compute the total number D of leaf directories.

(b) Compute the total dataset size V (in TB).

(c) A typical query reads *transactions for France in 2024, between March 1 and March 7*. Compute the partition-pruning ratio ρ_{read}.

(d) How much data is read at the partition level?

(a) $D = c_1 \cdot c_2 \cdot c_3 \cdot c_4 = 12 \times 5 \times 12 \times 31 = 22320$ directories

(b) $V_{\text{compressed}} = D \times 2\text{GB} = 22320 \times 2 = 44640 \text{ GB} = 44.64 \text{ TB}$

(c) $\rho_{\text{read}} = \frac{1}{12} \times \frac{1}{5} \times \frac{1}{12} \times \frac{7}{31} = \frac{7}{22320} \approx 0.0003136$

(d) $\text{Data read} = \rho_{\text{read}} \cdot V = \frac{7}{22320} \times 44640 \text{ GB} = 14\text{GB}$

Part B. Small-file problem. S3 has fixed per-file overhead $\tau = 100$ ms and streaming throughput $\beta = 200$ MB/s per request thread.

(e) Compute the optimal average file size s_{opt}.

(f) For the query in (c), how many files F must be read with the *current* layout of 200 files per day-partition?

(g) Compute the total read time T_{read} for the query under the current layout.

(h) The team runs a compaction job that rewrites each day-partition as a single 2 GB file. Compute the new total read time. What speed-up is achieved?

(e) $S_{opt} = \tau \cdot \beta = 100\,ms \times 200\,MB/s = 0.1s \times 200 MB/s = 20 MB$

(f) The query touches 7 day-partitions.

$$F = 7\,days \times 200\,files/day = 1400\,files$$

(g) $B = 14\,GB = 14000\,MB$

$$T_{read} = F \cdot \tau + \frac{B}{\beta} = 1400 \times 0.1 + \frac{14000\,MB}{200\,MB/s} = 140 + 70\,s = 210\,seconds$$

(h) New $F = 7$ files

$$T_{\text{read}}^{\text{new}} = (7 \times 0.1\text{s}) + \frac{14000\ \text{MB}}{200\ \text{MB/s}} = 0.7 + 70\text{ s} = 70.7\ \text{seconds}$$

Part C. The high-cardinality trap. The company is asked to add `user_id` (cardinality 10 million) as a partition column, between `country` and `year`.

(i) Compute the new total number of leaf directories.

(j) If the dataset size remains V as in (b), what is the average file size in each leaf directory, assuming the same number of files per leaf? Comment.

(k) Briefly justify mathematically why the proposal must be **rejected**.

(i) $D_{new} = 22320 \times 10000000 = 2.232 \times 10^{11}$ directories.

(j) Data per leaf $= \frac{44640\ \text{GB}}{2.232 \times 10^{11}} = 0.0000002\ \text{GB} = 200$ bytes

If split into 200 files. each file would mathematically average 1 byte.

The data is fragmented to an absurb degree. the payload is effectively zero, leaving nothing but system overhead and file metadata.

(K) the average file size s becomes vastly smaller than s_{opt}

$$1 \text{ byte} << 20 \text{ MB}$$

the total read time formula $T_{read} = F_t + B/\beta$

will be completely domained by the fixed metadata overhead F_t

Any query would spend >99.9% of its time just opening tiny, nearly empty files on S3, causing a severe performance collapse.